



COURSE MATERIAL

Walk the Parser

CMSC 124 Activity 7

Prepared by: Rene Andre Bedonia Jocsing

Published: September 17, 2026

Deadline: September 18, 2026

A **recursive-descent parser** is a top-down parser with one function for each grammar rule. **Top down** means it begins at the start rule and asks which grammar alternative can consume the next tokens. Today you'll keep three traces side by side: the next unread token, the active parser calls, and the syntax tree built so far.

The parser below is pseudocode. It's written to be read. Every helper it calls is defined on this page. Laboratory Activity 2 is where this turns into code, since the recursive descent and the synchronization you trace today are what that laboratory asks you to implement.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. You may discuss with classmates and the instructor throughout. This activity is worth 20 points. Twelve points reward parser constructions, and eight reward the reasoning behind them.

Grammar and Parser Contract

Use this Extended Backus-Naur Form (EBNF) grammar:

```
1 expression ::= term
2 term       ::= factor { ( PLUS | MINUS ) factor }
3 factor     ::= primary { ( STAR | SLASH ) primary }
4 primary    ::= NUMBER | LEFT_PAREN expression RIGHT_PAREN
```

The bottom three rules are Activity 4's levels under the same labels. `term` handles addition and subtraction. Below it, `factor` handles multiplication and division, while `primary` handles a number or a parenthesized expression.

A **nonterminal** is a named grammar rule such as `term`. The parser translates each rule almost literally. It turns nonterminals into same-label functions, references into calls, tokens into consuming checks, alternatives into branches, and EBNF repetitions into loops.

Start with the smallest rule. In `primary`, the `NUMBER` alternative becomes the first `if`. The parenthesized alternative consumes `LEFT_PAREN`, calls `expression()` for the nested nonterminal, and then requires `RIGHT_PAREN`. The function returns the syntax-tree node built by the alternative that matched.

`peek()` returns the next token without consuming it. `match(types...)` takes one or more token types, consumes the next token when it's any of them, and answers true or false. The three dots say the call accepts one or more types, so `match(PLUS, MINUS)` asks one question about two operators. `previous()` returns the token `match` last consumed. `consume(T)` requires and consumes type `T`. If that type isn't next, it reports an error and fails the current expression.

A token is **consumed** once the parser advances past it. The **next unread token** is the token at the parser's current position. The **call stack** is the ordered set of parser functions that have started

but haven't returned. EOF means **end of file**. It marks the end of the token stream. These expression functions leave it unread.

Read `term ::= factor { ( PLUS | MINUS ) factor }` as “term produces a factor followed by zero or more plus-or-minus factor pairs.” *Read* `primary()` as “call primary.”

```

1  expression(): // expression ::= term
2      return term()
3
4  term():      // term ::= factor { ( PLUS | MINUS ) factor }
5      node = factor()
6      while match(PLUS, MINUS):
7          node = Binary(node, previous(), factor())
8      return node
9
10 factor():    // factor ::= primary { ( STAR | SLASH ) primary }
11     node = primary()
12     while match(STAR, SLASH):
13         node = Binary(node, previous(), primary())
14     return node
15
16 primary():   // primary ::= NUMBER | LEFT_PAREN expression RIGHT_PAREN
17     if match(NUMBER): return Number(previous token)
18     if match(LEFT_PAREN):
19         node = expression()
20         consume(RIGHT_PAREN); return Group(node)
21     report an error at peek() and fail this expression

```

An **abstract syntax tree** (AST) keeps meaningful structure and omits grammar bookkeeping. We'll write `Binary(left, operator, right)`, `Number(value)`, and `Group(inner)`. A group node has one child and records that the source wrapped it in parentheses, so a printer walking the tree can put those parentheses back where the programmer wrote them.

Read `Binary(left, operator, right)` as “a binary node with this left child, operator, and right child.” *Read* `Group(inner)` as “a group node holding this one child.”

For this grammar, `expression()` only returns `term()`. That short function gives every caller one stable entry point for a complete expression. A later grammar can place assignment or equality between `expression` and `term` without changing `primary()` or any caller outside the parser.

Worked Example for Checkpoint 1

Suggested time: 28 minutes, including the worked example.

Parse `NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF`.

Begin with a fresh parser. No parser function is active. The next unread token is `NUMBER(4)`. No AST node exists. The parser enters the start rule by calling `expression()`.

Follow the frames in order. Calls, matches, failed matches, tree updates, and returns each get their own figure. The complete token stream appears under every figure heading. A bold token is consumed in that figure. When no token is bold, the parser position doesn't move.

Descend to the first operand

Each frame keeps the complete token stream, grammar rule, and parser function in view. Bold marks the token consumed and the line running in that frame.

expression() calls *term()*

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

No token is consumed.

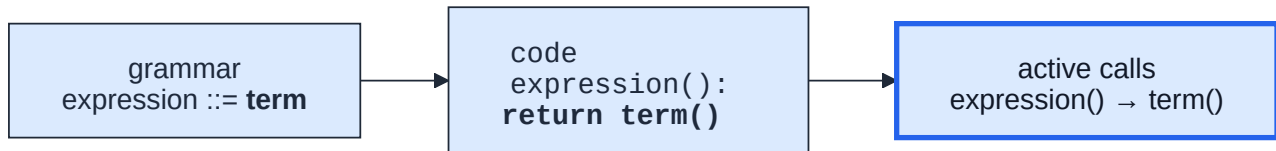


Figure 1: expression() calls term()

term() calls *factor()*

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

term() requests its first operand. No token is consumed.

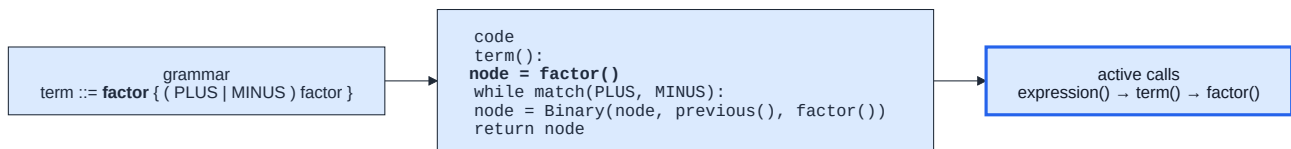


Figure 2: term() calls factor()

factor() calls *primary()*

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

factor() requests its first operand. No token is consumed.

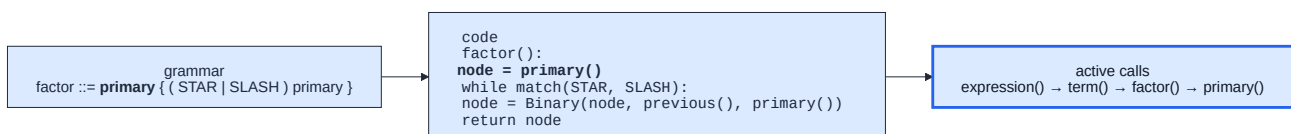


Figure 3: factor() calls primary()

Match the first operand, then unwind to factor()

The parser reaches the bottom rule. Watch the first match, then watch `primary()` leave the call stack.

primary() matches NUMBER(4)

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

The NUMBER alternative matches. `primary()` consumes the token and creates `Number(4)`.

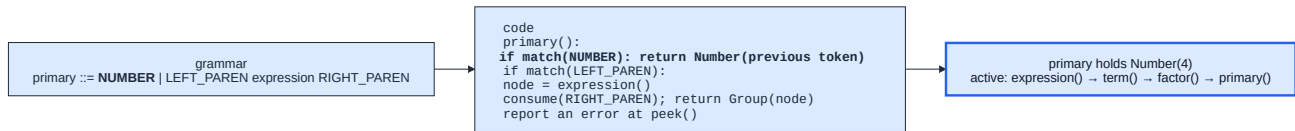


Figure 4: `primary()` matches `NUMBER(4)`

primary() returns Number(4) to factor()

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`primary()` leaves the call stack. `factor()` receives `Number(4)` as its first operand.

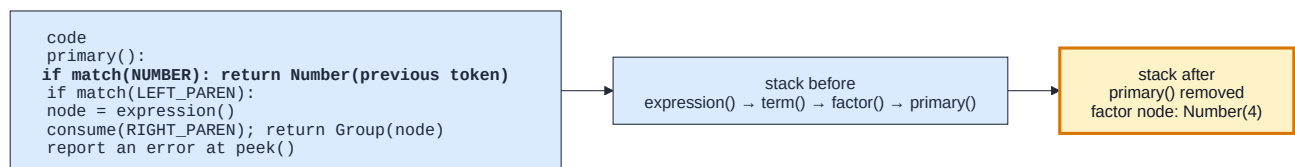


Figure 5: `primary()` returns `Number(4)` to `factor()`

factor() tests for a multiplicative operator and fails

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`match(STAR, SLASH)` sees PLUS and answers false. The loop body never runs and no token moves.

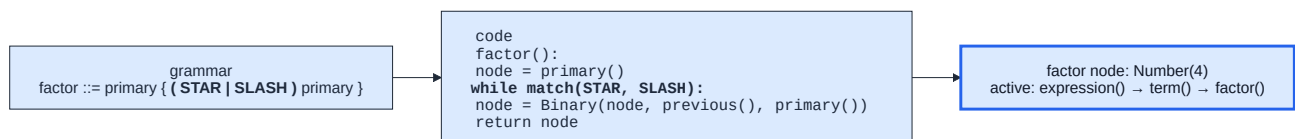


Figure 6: `factor()` tests for a multiplicative operator and fails

Return to term() and take the operator

Control returns to `term()`. The parser then takes **PLUS** and asks for the operand on its right.

factor() returns Number(4) to term()

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`factor()` leaves the call stack. `term()` receives `Number(4)` as its first operand.

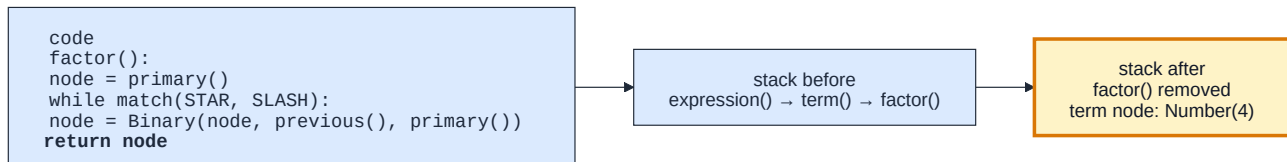


Figure 7: `factor()` returns `Number(4)` to `term()`

term() consumes PLUS

NUMBER(4) **PLUS** NUMBER(5) STAR NUMBER(2) EOF

The repetition matches. `term()` saves the operator before it requests the right operand.

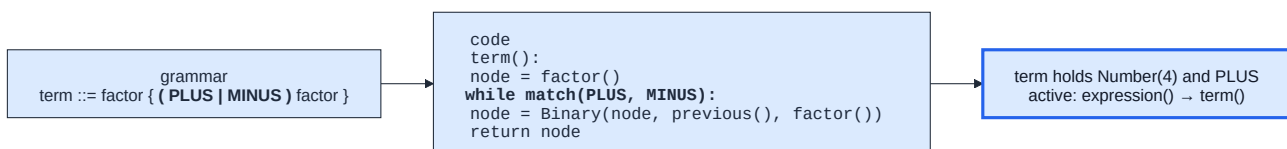


Figure 8: `term()` consumes **PLUS**

term() calls factor() for the right operand

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

The loop body requests the operand to the right of **PLUS**. No token is consumed.

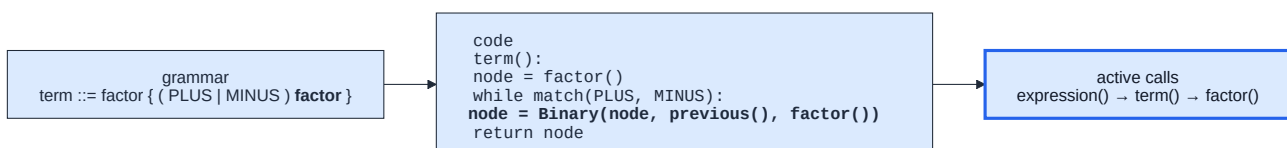


Figure 9: `term()` calls `factor()` for the right operand

Descend to the second operand

The parser descends again for the operand after PLUS. The same call pattern repeats.

factor() calls primary()

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

The new `factor()` requests its first operand. No token is consumed.

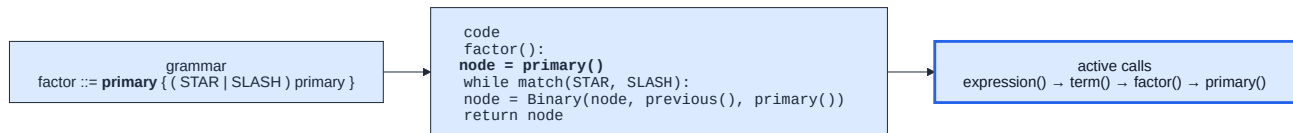


Figure 10: `factor()` calls `primary()`

primary() matches NUMBER(5)

NUMBER(4) PLUS **NUMBER(5)** STAR NUMBER(2) EOF

The **NUMBER** alternative matches. `primary()` consumes the token and creates `Number(5)`.

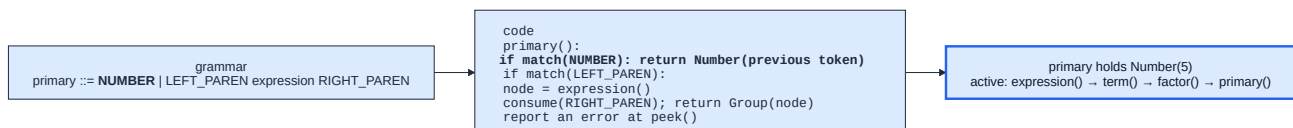


Figure 11: `primary()` matches `NUMBER(5)`

primary() returns Number(5) to factor()

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`primary()` leaves the call stack. `factor()` receives `Number(5)` as its left operand.

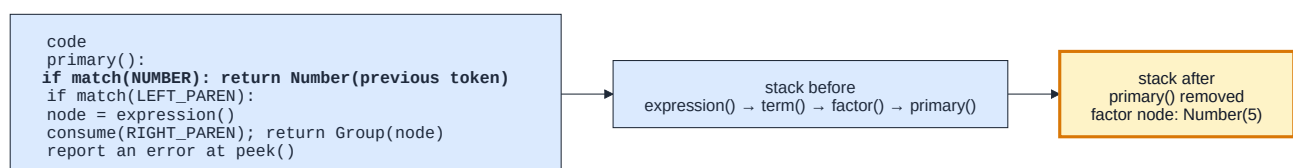


Figure 12: `primary()` returns `Number(5)` to `factor()`

Take the multiplicative operator and its operand

The loop inside `factor()` takes `STAR`. It then asks for the operand on the right of `STAR`.

factor() consumes STAR

NUMBER(4) PLUS NUMBER(5) **STAR** NUMBER(2) EOF

The repetition matches. `factor()` saves the operator before it requests the right operand.

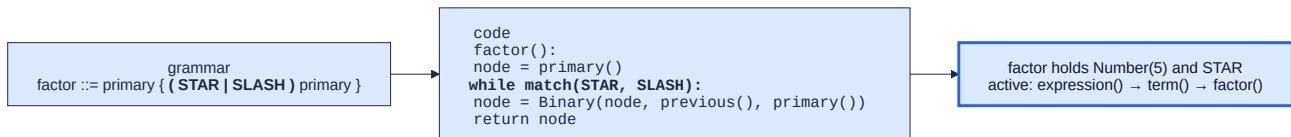


Figure 13: `factor()` consumes `STAR`

factor() calls primary() for the right operand

NUMBER(4) PLUS NUMBER(5) **STAR** NUMBER(2) EOF

The loop body requests the operand to the right of `STAR`. No token is consumed.

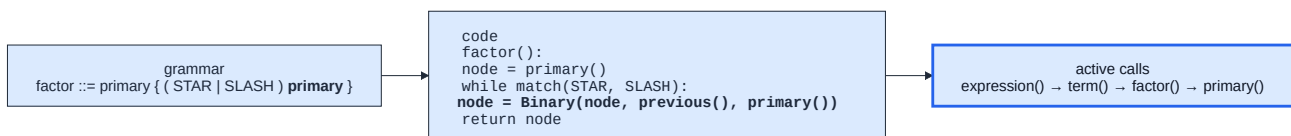


Figure 14: `factor()` calls `primary()` for the right operand

primary() matches NUMBER(2)

NUMBER(4) PLUS NUMBER(5) **STAR** **NUMBER(2)** EOF

The `NUMBER` alternative matches. `primary()` consumes the token and creates `Number(2)`.

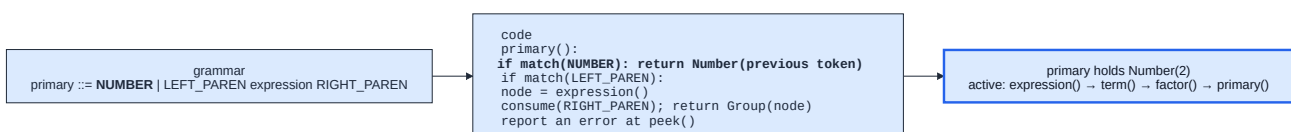


Figure 15: `primary()` matches `NUMBER(2)`

Build the multiplication subtree

Both multiplication operands are ready. `factor()` joins them, then tests for another multiplicative operator.

primary() returns Number(2) to factor()

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`primary()` leaves the call stack. `factor()` now holds both multiplication operands.

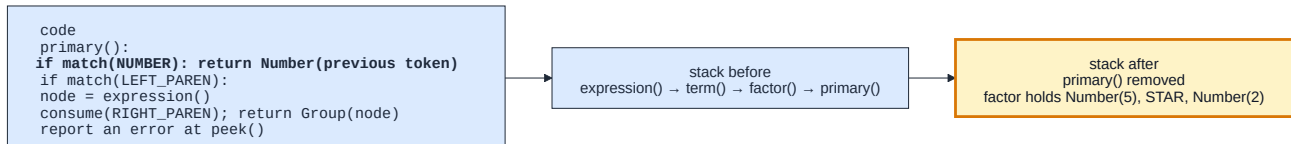


Figure 16: `primary()` returns Number(2) to `factor()`

factor() builds the multiplication subtree

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

The loop body joins the saved left operand, operator, and returned right operand. No token is consumed.

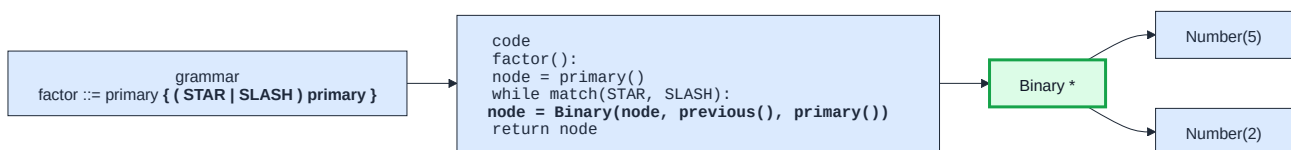


Figure 17: `factor()` builds the multiplication subtree

factor() tests for a second multiplicative operator and fails

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`match(STAR, SLASH)` sees EOF and answers false. The loop ends and no token moves.

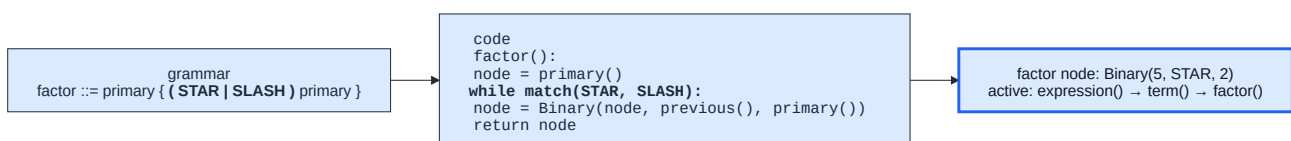


Figure 18: `factor()` tests for a second multiplicative operator and fails

Build the addition tree

The multiplication subtree moves up to `term()`. Then `term()` joins its saved parts into the addition tree.

factor() returns the multiplication subtree to term()

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`factor()` leaves the call stack. `term()` receives the multiplication subtree as its right operand.

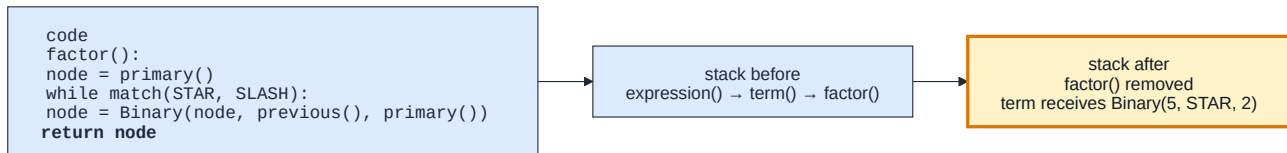


Figure 19: `factor()` returns the multiplication subtree to `term()`

term() builds the addition tree

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

The loop body joins its saved parts with the multiplication subtree. No token is consumed.

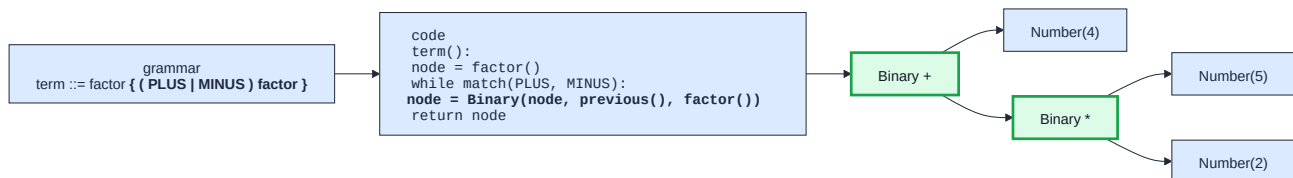


Figure 20: `term()` builds the addition tree

term() tests for a second additive operator and fails

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`match(PLUS, MINUS)` sees EOF and answers false. The loop ends and no token moves.

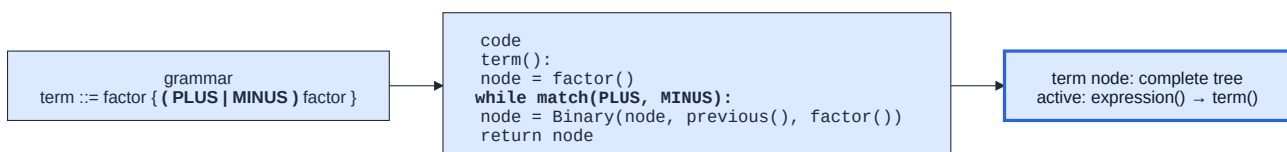


Figure 21: `term()` tests for a second additive operator and fails

Unwind to the start rule

The finished tree leaves `term()` and reaches `expression()`. The parse ends.

term() returns the complete tree to expression()

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`term()` leaves the call stack. `expression()` receives the finished tree.

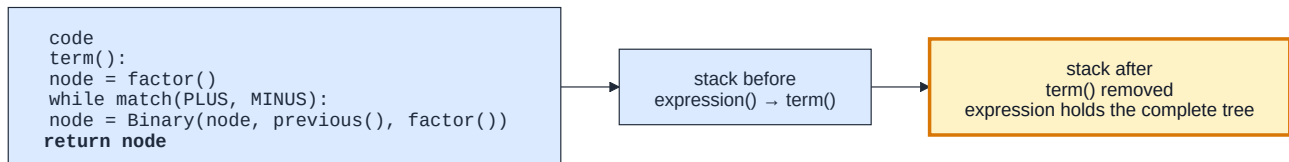


Figure 22: `term()` returns the complete tree to `expression()`

expression() returns the complete AST

NUMBER(4) PLUS NUMBER(5) STAR NUMBER(2) EOF

`expression()` returns the tree. EOF remains unread because no expression rule consumes it.

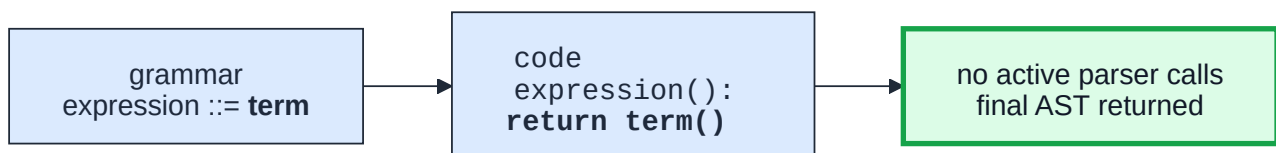


Figure 23: `expression()` returns the complete AST

Only `match` and `consume` advance the token position. Calls and returns change the active stack, while assignments to `node` change the AST fragment. Use the same after-consumption convention in your trace.

The multiplication becomes a complete subtree before the addition can be built. `term()` asks `factor()` for its entire right operand, so `term()` must wait until `factor()` returns. The final AST is `Binary(Number(4), PLUS, Binary(Number(5), STAR, Number(2)))`.

Checkpoint 1: Keep Three Traces (8 Points)

Running Token Stream

Parse this stream. The numbers carry their lexemes.

1 NUMBER(6) MINUS NUMBER(2) STAR NUMBER(3) EOF

Keep this contract beside you while you trace. You don't need to flip back to the front page.

```

1 expression ::= term
2 term       ::= factor { ( PLUS | MINUS ) factor }
3 factor     ::= primary { ( STAR | SLASH ) primary }
4 primary    ::= NUMBER | LEFT_PAREN expression RIGHT_PAREN

```

```

1  expression(): // expression ::= term
2      return term()
3
4  term():      // term ::= factor { ( PLUS | MINUS ) factor }
5      node = factor()
6      while match(PLUS, MINUS):
7          node = Binary(node, previous(), factor())
8      return node
9
10 factor():    // factor ::= primary { ( STAR | SLASH ) primary }
11     node = primary()
12     while match(STAR, SLASH):
13         node = Binary(node, previous(), primary())
14     return node
15
16 primary():   // primary ::= NUMBER | LEFT_PAREN expression RIGHT_PAREN
17     if match(NUMBER): return Number(previous token)
18     if match(LEFT_PAREN):
19         node = expression()
20         consume(RIGHT_PAREN); return Group(node)
21     report an error at peek() and fail this expression

```

`match(types...)` consumes the next token when its type is listed and answers true. `previous()` returns the token match last consumed. `consume(T)` requires type T at `peek()` and consumes it. Only `match` and `consume` move the token position.

Use the parser contract from this activity. `term()` first calls `factor()`. Its loop combines each PLUS or MINUS right operand with the tree already saved in `node`. `factor()` does the same for each STAR or SLASH right operand after it calls `primary()`. `primary()` consumes a number or a parenthesized expression. Record the next unread token after every consumption.

Read NUMBER(6) MINUS NUMBER(2) STAR NUMBER(3) EOF as “number six, minus, number two, star, number three, end of file.” Read `term()` and `factor()` as “call term” and “call factor.”

1. (Construction: 5 points) Make one trace entry for every consumed token. Each entry must show the token’s index and type, the active call stack, the AST fragment returned or extended, and the next unread token. Finish by writing the complete AST.

2. (Explanation: 3 points) Explain why `factor()` consumes the multiplication before the outer `term()` can finish the subtraction. Then explain why the loop in `term()` makes a chain such as $8 - 3 - 2$ associate left.

Why Left Recursion Stalls

Take $1 + 2$ with this addition grammar. You’ll repair the same addition rule with PLUS in the worked example, then apply it to subtraction in the checkpoint.

See this old rule:

```

1  chain ::= chain PLUS item | item

```

A literal `chain()` picks the first alternative and calls `chain()` again before it looks for PLUS or `item`. Each call sees the same unread token NUMBER(1) and consumes nothing. The stack grows by one frame per call and never reaches `item`.

A rule is **left recursive** when its first alternative starts with the rule itself. Recursive descent can’t run that rule directly because the recursive call comes before any consumption.

Watch the stalled calls pile up

Each frame keeps the same unread token in view. No frame consumes input, so the stack only grows.

First chain() call sees NUMBER(1)

NUMBER(1) PLUS NUMBER(2) PLUS NUMBER(3) EOF

No token is consumed. The first chain() picks the recursive alternative and calls itself.

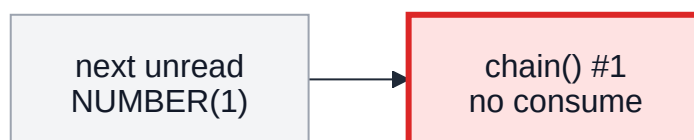


Figure 24: First chain() call sees NUMBER(1)

Second chain() call sees the same NUMBER(1)

NUMBER(1) PLUS NUMBER(2) PLUS NUMBER(3) EOF

No token is consumed. The second call repeats the same choice. The stack now holds two frames.

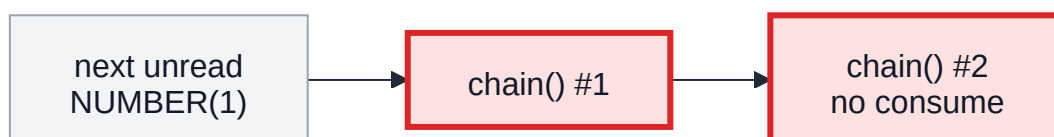


Figure 25: Second chain() call sees the same NUMBER(1)

Third chain() call still sees NUMBER(1)

NUMBER(1) PLUS NUMBER(2) PLUS NUMBER(3) EOF

No token is consumed. A third frame starts the same way. The descent never reaches item.

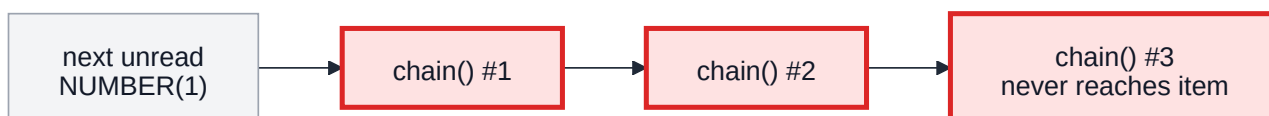


Figure 26: Third chain() call still sees NUMBER(1)

This rewritten grammar preserves accepted token sequences. It still accepts one `item` followed by zero or more `PLUS item` pairs. The accumulator assignment preserves left-associated AST construction. It carries the tree built so far into the next loop pass as the left child.

Use this replacement rule, repairing the defect:

```
1 chain ::= item { PLUS item }
```

And its corresponding pseudocode:

```
1 chain():
2     node = item()
3     while match(PLUS):
4         operator = previous()
5         right = item()
6         node = Binary(node, operator, right)
7     return node
```

How the replacement rule repairs the stall

The old and replacement rules are successive versions. The replacement reads one item before its loop, so every later pass starts from input already consumed.

Replace the rule, then map its parts to code

`chain ::= chain PLUS item | item` is the old rule to remove. `chain ::= item { PLUS item }` is the replacement to use, the repaired version. The first `item()` call returns the value stored in `node`, which becomes the seed for the loop.

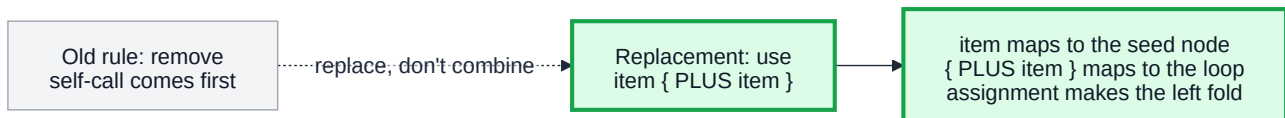


Figure 27: Replace the rule, then map its parts to code

Worked Example for Checkpoint 2

Suggested time: 17 minutes, including the worked example.

For $1 + 2 + 3$, the first `item()` call returns `Number(1)`. The first loop pass consumes `PLUS` and `Number(2)` and builds `Binary(Number(1), PLUS, Number(2))`. The second pass carries that whole tree as its left child and builds `Binary(Binary(Number(1), PLUS, Number(2)), PLUS, Number(3))`. Carrying `node` forward creates left grouping.

Fold 1 + 2 + 3 from the left

The accumulator node carries the previous tree into each pass. Each frame shows the whole tree so far.

Seed node, then run the first pass

NUMBER(1) PLUS NUMBER(2) PLUS NUMBER(3) EOF

node starts as `Number(1)`. The first pass uses that node as its left child and produces `Binary(Number(1), PLUS, Number(2))`.

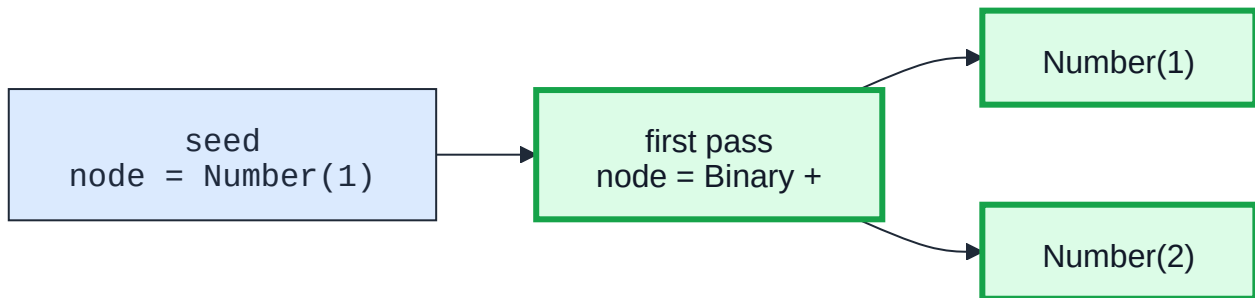


Figure 28: Seed node, then run the first pass

Second pass builds $((1 + 2) + 3)$

NUMBER(1) PLUS NUMBER(2) PLUS NUMBER(3) EOF

The pass carries the whole first tree as its left child. It builds `Binary(Binary(Number(1), PLUS, Number(2)), PLUS, Number(3))`.

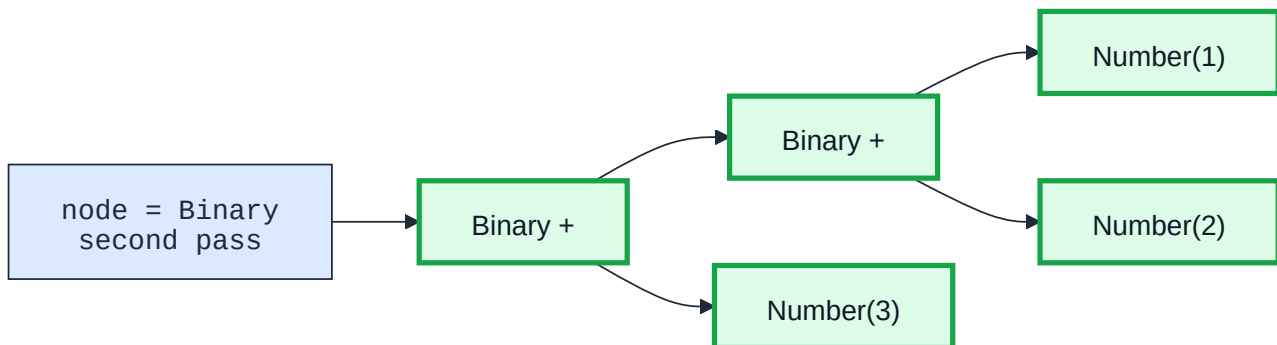


Figure 29: Second pass builds $((1 + 2) + 3)$

Checkpoint 2: Turn Recursion into Progress (6 Points)

Repair this exact left-recursive rule while preserving its subtraction chains and left grouping:

```
1 sum ::= sum MINUS product | product
```

1. (Construction: 4 points) Rewrite `sum` in EBNF without left recursion, then sketch the corresponding parser function. Its loop must consume one operator and one right operand per iteration.

2. (Explanation: 2 points) Explain why your rewrite accepts the same subtraction chains and preserves left associativity. Point to the assignment that carries the previous subtree into the next iteration.

Any rewrite must preserve the sequences, terminate, and associate left.

Transfer Through an Error

For this section, place the expression grammar inside a small program rule:

```
1 program ::= { expression SEMICOLON } EOF
```

Each semicolon ends one expression. If that expression fails, the program driver can skip it and try the expression after the semicolon.

Synchronization is error recovery that brings the parser back to a position where parsing can restart. It must repair two things. First, the failed expression calls must leave the call stack. Second, the parser must advance past bad input to a known boundary. Here, that boundary is SEMICOLON or EOF.

The scanner and parser answer different questions. The scanner turns the complete source into tokens before parsing begins. It accepts `)` because that character forms a valid RIGHT_PAREN token. The parser can still reject the token when the grammar requires NUMBER or LEFT_PAREN at its position.

Worked Example for Checkpoint 3

Suggested time: 15 minutes, including the worked example.

The scanner turns `4 + ); 2;` into `NUMBER(4) PLUS RIGHT_PAREN SEMICOLON NUMBER(2) SEMICOLON EOF`. The driver starts the first expression at `NUMBER(4)`. After `term()` consumes `PLUS`, it calls `factor()`, which calls `primary()` for the missing right operand.

This is the decision that `primary()` makes:

```
1 primary():    // primary ::= NUMBER | LEFT_PAREN expression RIGHT_PAREN
2   if match(NUMBER): return Number(previous token)
3   if match(LEFT_PAREN):
4       node = expression()
5       consume(RIGHT_PAREN); return Group(node)
6   report an error at peek() and fail this expression
```

At this call, `peek()` is `RIGHT_PAREN`. It isn't `NUMBER`, so the first `match` answers false without consuming it. It isn't `LEFT_PAREN` either, so the second match also answers false. Execution then **falls through** to the final line. Falling through means that no earlier branch returned, so the function continues with the next statement. `primary()` reports the unread `RIGHT_PAREN` and fails the expression.

That failure returns through `factor()`, `term()`, and `expression()` until control reaches the program driver. These returns clear the failed expression calls from the stack. They don't consume `RIGHT_PAREN`. The partial `Number(4)` is abandoned because it never became a complete expression.

The following figures begin at that exact state. Bold means the parser advances past that token in the figure. Orange marks discarded input. Amber marks the consumed boundary. Blue marks the next expression's input. Gray marks an abandoned partial result.

Move the token position to the next expression

After the call stack resets, recovery discards input to the semicolon and starts a separate expression after it.

Discard the bad token and consume the boundary

NUMBER(4) PLUS **RIGHT_PAREN** SEMICOLON EOF

`discard()` advances past `RIGHT_PAREN` without creating a node. `match(SEMICOLON)` then consumes the boundary, so `NUMBER(2)` becomes the next unread token.

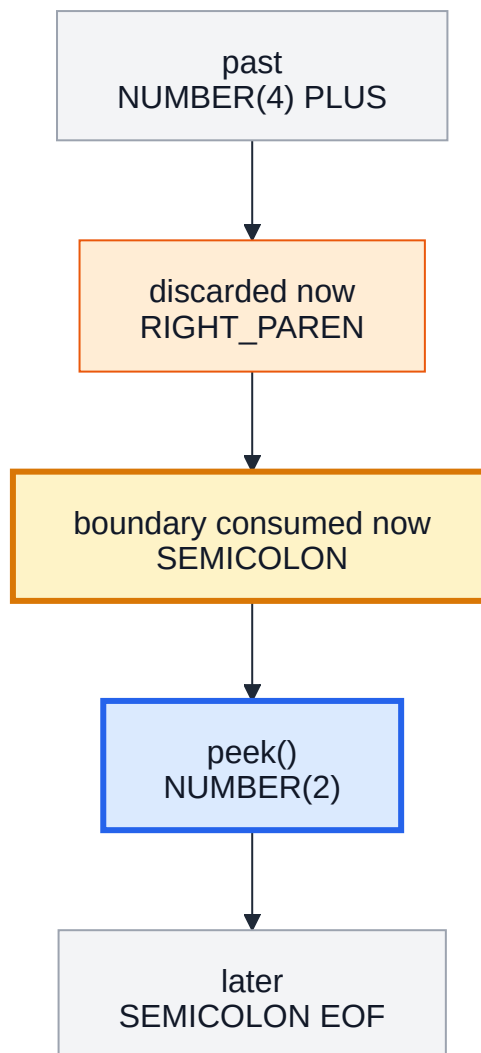


Figure 30: Discard the bad token and consume the boundary

Start a separate expression after the boundary

Recovery has ended. The program driver now starts a new parse at the next unread token.

Parse the next expression and keep only its AST

NUMBER(4) PLUS RIGHT_PAREN SEMICOLON **NUMBER(2) SEMICOLON** EOF

A new `expression()` consumes `NUMBER(2)`. The driver consumes its semicolon and keeps `Number(2)`. The failed expression remains separate and contributes no completed AST.

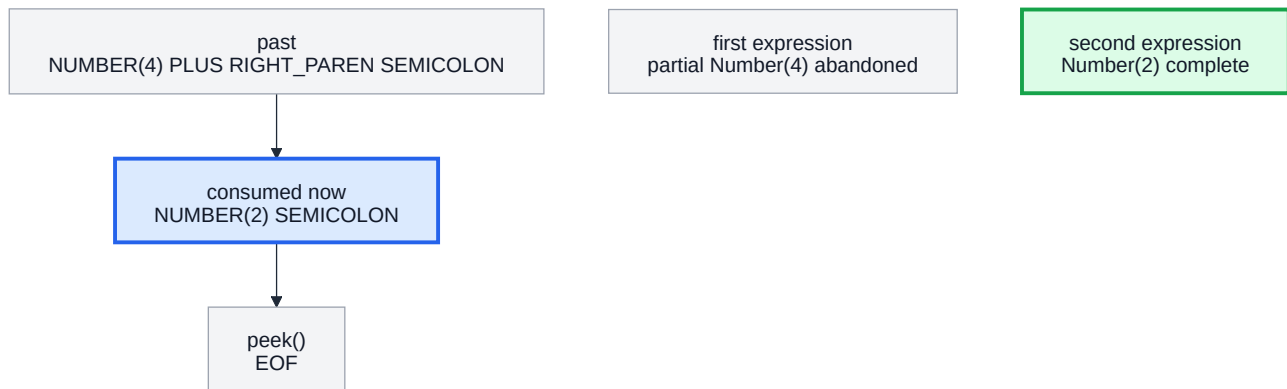


Figure 31: Parse the next expression and keep only its AST

Recovery doesn't invent a right operand or turn `)` into one. The next parse starts separately at `NUMBER(2)` and produces the only completed AST, `Number(2)`.

This language-neutral driver shows where recovery runs. `discard()` advances past one token without creating an AST node.

```

1  program():
2      while peek() is not EOF:
3          attempt expression() followed by consume(SEMICOLON)
4          if the attempt succeeds:
5              keep the completed expression AST
6          otherwise:
7              synchronize()
8
9  synchronize():
10     while peek() is neither SEMICOLON nor EOF:
11         discard()
12     if match(SEMICOLON):
13         return to program() at the token after the boundary
14     return to program() at EOF
  
```

At EOF, the driver's loop stops. It doesn't call `expression()` again. After a semicolon, the loop starts a new attempt at the next token.

Checkpoint 3: Recover and Continue (6 Points)

Running Source

Recover from this source:

```

1  6 * (2 + ); 9 - 1;
  
```

Report the unexpected token. Let the failed expression calls return to the program driver. Discard tokens before the next SEMICOLON, consume that boundary, and restart at the token after it. If recovery reaches EOF, stop instead.

1. (Construction: 3 points) Tokenize the input. Mark the active call stack and rejected token at the first error. Then mark the discarded tokens, the consumed boundary, and the first token of the next expression. Draw only that expression's completed AST.

2. (Explanation: 3 points) Explain why the scanner accepts `)` but `primary()` rejects it here. Then explain why synchronization abandons the first expression, stops at the semicolon, and resumes at 9.